

TITLE : FAULT DIAGNOSIS IN INDUSTRIAL SYSTEM

TEAM MEMBERS

K.PRAGNYA – 2520030430

G.SAI LASYA – 2520030538

V.KEATHANA - 2520090236

INTRODUCTION

- INDUSTRIAL SYSTEMS SUCH AS MANUFACTURING PLANTS, POWER STATIONS, AND AUTOMATED PRODUCTION LINES RELY HEAVILY ON CONTINUOUS MONITORING TO ENSURE SAFE AND EFFICIENT OPERATION. UNEXPECTED EQUIPMENT FAILURES CAN LEAD TO PRODUCTION LOSSES, INCREASED MAINTENANCE COSTS, AND SAFETY HAZARDS. THEREFORE, THE EARLY DETECTION AND DIAGNOSIS OF FAULTS IN INDUSTRIAL MACHINERY HAVE BECOME CRITICAL FOR IMPROVING SYSTEM RELIABILITY AND REDUCING DOWNTIME.
- THE PROJECT "**FAULT DIAGNOSIS IN INDUSTRIAL SYSTEMS USING PYTHON**" AIMS TO DEVELOP AN INTELLIGENT DIAGNOSTIC SYSTEM THAT ANALYSES SENSOR DATA SUCH AS TEMPERATURE, VIBRATION, PRESSURE, AND ELECTRICAL CURRENT TO IDENTIFY POTENTIAL FAULTS IN INDUSTRIAL EQUIPMENT. THE SYSTEM INTEGRATES MULTIPLE ARTIFICIAL INTELLIGENCE (AI) TECHNIQUES INCLUDING RULE-BASED KNOWLEDGE REPRESENTATION, SEARCH ALGORITHMS, CONSTRAINT SATISFACTION, PROBABILISTIC REASONING, AND DECISION-MAKING MECHANISMS TO PROVIDE ACCURATE FAULT DIAGNOSIS.
- BY IMPLEMENTING THIS PROJECT IN PYTHON, THE SYSTEM DEMONSTRATES HOW AI TECHNIQUES CAN BE APPLIED TO REAL-WORLD INDUSTRIAL APPLICATIONS. THE PROPOSED FAULT DIAGNOSIS SYSTEM NOT ONLY DETECTS ABNORMAL OPERATING CONDITIONS BUT ALSO ASSISTS IN MAINTENANCE DECISIONS, THEREBY IMPROVING OPERATIONAL EFFICIENCY, RELIABILITY, AND SAFETY IN INDUSTRIAL ENVIRONMENTS.
- OVERALL, THIS PROJECT SERVES AS A PRACTICAL APPLICATION OF ARTIFICIAL INTELLIGENCE CONCEPTS IN INDUSTRIAL AUTOMATION AND SHOWCASES THE EFFECTIVENESS OF HYBRID AI SYSTEMS IN SOLVING COMPLEX DIAGNOSTIC PROBLEMS.

Python Code

```
"""
PROJECT: Fault Diagnosis in Industrial Systems
CO1 - Knowledge representation, states, rules
CO2 - Search algorithms (BFS)
CO3 - CSP constraints
CO4 - Decision making using utility
CO5 - Bayesian probability reasoning
CO6 - Hybrid AI system
"""
from collections import deque
from dataclasses import dataclass
from typing import Dict, List

@dataclass
class MachineState:
    temperature: float
    vibration: float
    pressure: float
    current: float

fault_rules = {
    "Bearing Fault": lambda s: s.vibration > 7,
    "Overheating": lambda s: s.temperature > 85,
    "Leakage": lambda s: s.pressure < 20,
    "Electrical Fault": lambda s: s.current > 50
}

def bfs_diagnosis(state):
    queue = deque([state])
    visited = set()
    while queue:
        current = queue.popleft()
        key = (current.temperature, current.vibration,
              current.pressure, current.current)
        if key in visited:
            continue
        visited.add(key)
        faults = []
        for fault, rule in fault_rules.items():
            if rule(current):
                faults.append(fault)
        return faults
    return ["Normal"]
```

```
def check_constraints(state):
    violations = []
    if state.temperature > 90 and state.current > 55:
        violations.append("Temperature-Current constraint violated")
    if state.vibration > 8 and state.pressure < 25:
        violations.append("Vibration-Pressure constraint violated")
    return violations

def bayes_fault_probability(temp):
    prior_fault = 0.1
    likelihood = 0.8 if temp > 85 else 0.2
    evidence = 0.3
    return round((likelihood * prior_fault) / evidence, 3)

def utility(state):
    score = 100
    score -= abs(state.temperature - 70)
    score -= state.vibration * 2
    score -= abs(state.pressure - 30)
    return score

def diagnose(state):
    faults = bfs_diagnosis(state)
```

```
constraints=check_constraints(state)
probability=bayes_fault_probability(state.temperature)
utility_score=utility(state)
print(faults, constraints, probability, utility_score)

machine=MachineState(95,8.5,18,60)
diagnose(machine)
```

Sample Output

----- INDUSTRIAL DIAGNOSIS -----

Detected Faults:

['Bearing Fault', 'Overheating', 'Leakage', 'Electrical Fault']

Constraint Violations:

- Temperature-Current constraint violated
- Vibration-Pressure constraint violated

Fault Probability: 0.267

Utility Score: 16.0

Decision: Continue Operation

Output

Clear

----- INDUSTRIAL DIAGNOSIS -----

Detected Faults: ['Bearing Fault', 'Overheating', 'Leakage', 'Electrical Fault']

Constraint Violations:

- Temperature-Current constraint violated
- Vibration-Pressure constraint violated

Fault Probability: 0.267

Utility Score: 46.0

Decision: Continue Operation

=== Code Execution Successful ===

The image features a light gray background with a subtle gradient. In the corners, there are several realistic-looking water droplets of various sizes, some with highlights and shadows, giving them a three-dimensional appearance. The text "THANK YOU" is centered in a bold, black, sans-serif font.

THANK YOU